FREE FIELD GUIDE · @sabaautomations

# Can your AI actually read your files?

What I measured when I changed only the way a small local AI read Word tables, spreadsheets and scanned PDFs. Same model. Same questions. **23 of 31 became 31 of 31.**

**23 → 31**

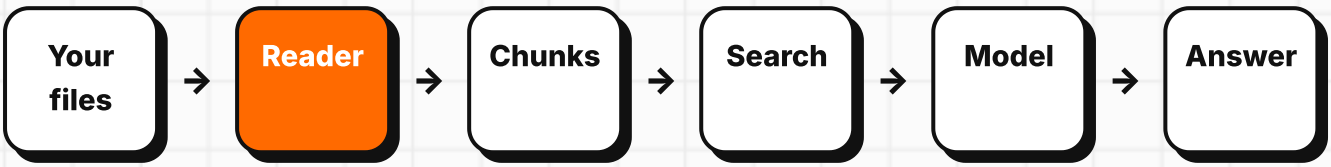
**correct answers, same free model on a laptop CPU**

**Project: github.com/SabaAutomation/multidoc-brain (MIT licence)**

Built in public · October 2026

THE PROBLEM

# Most wrong answers are decided before the model reads anything.




Demos use clean text. Real folders hold Word tables, spreadsheets, CSVs and scans. The **reader** turns those into text. Whatever it throws away, the model can never see, and it will answer anyway.

### Three ways a plain-text reader loses information

**Word tables disappear.** Paragraphs come through, the table with the price list does not.

**Spreadsheet rows lose their meaning.** A row becomes `RB-310 Ridge Boots 154 8`. The right numbers are found, but nothing says which one is the price.

**Scans are blank.** A scanned PDF is a picture. With no text layer there is nothing to read.

THE EXPERIMENT

# Four ways to read the same eight files.

Same documents, same 34 questions, same free local model (`qwen2.5-coder:7b` through Ollama, on a laptop CPU, no paid API). The only thing that changes is the reading level.

| Level | What the reader does |
|---|---|
| **P0** | Plain text only. Word tables dropped, spreadsheet rows as bare values, scans unreadable. |
| **P1** | Table-aware. Each row becomes `Header: value \| Header: value`. |
| **P2** | P1 plus a label on every chunk: `[file \| sheet / page / table]`. |
| **P3** | P2 plus OCR (Tesseract) for pages with no text layer. |

### The test set

- 8 invented files for a made-up shop: Word handbook with a table, meeting notes, PDF policy, two spreadsheets, a CSV, a Markdown FAQ and one scanned PDF.
- 34 questions with known answers: 31 answerable, 3 **traps** where the answer is in no file and the right reply is `NOT FOUND`.
- Scoring is plain string matching against the known answer. No model grades the model.

**Two separate measurements.** Retrieval: did the answer reach the model? (no LLM involved). Answers: did the model then get it right? Keeping them apart shows *where* a failure happens.

THE RESULTS

# Same model. 23 → 31 of 31.




| File type | P0 | P1 | P2 | P3 |
|---|---|---|---|---|
| Prose (pdf / docx / md) | 15/15 | 15/15 | 15/15 | 15/15 |
| Table inside a docx | **0/3** | **3/3** | 3/3 | 3/3 |
| Spreadsheet (xlsx) | 5/7 | 5/7 | **7/7** | 7/7 |
| CSV | 3/3 | 3/3 | 3/3 | 3/3 |
| Scanned PDF | **0/3** | **0/3** | **0/3** | **3/3** |
| **All answerable** | **23/31** | **26/31** | **28/31** | **31/31** |

**Traps:** the three unanswerable questions were refused correctly (`NOT FOUND`) 3/3 at every level. The model did not guess.

**Retrieval only (hit@4, no LLM):** 25/31, 28/31, 28/31, 31/31. A table or scan the reader drops cannot be retrieved, so no model can answer it.

HONEST LIMITS

# What this does not prove.

- **Small and synthetic.** 8 invented files and 34 questions. A difference of one or two questions is within noise.
- **One run per cell.** The spreadsheet gap (5/7 at P0 and P1, 7/7 at P2) is not explained yet. The right answer was in front of the model and it still missed two. Repeat it before trusting it.
- **Keyword search only.** Retrieval is BM25. Embeddings could behave differently.
- **One small model.** Other models may do better or worse at each level.
- **Cleaner than real life.** Real client files are messier than these.

**What it does show:** reading tables and scans properly decides whether the answer even reaches the model. That part is deterministic, and you can check it yourself.

### Why publish the weak spots?

A result you cannot check is a claim, not a result. The test set, the scoring code and the raw answers are in the repo so anyone can rerun them.

THE FIX LIST

# Six changes you can make to your own RAG this week.

**1 · Keep headers with every row.** Turn table rows into `Header: value | Header: value` before chunking.

**2 · Label every chunk with its source.** `[file | sheet / page / table]` lets the model, and you, see where an answer came from.

**3 · OCR pages with no text layer.** Tesseract is free. On Windows: `winget install UB-Mannheim.TesseractOCR`.

**4 · Keep tables in document order with their caption.** The paragraph before a table usually says what it is.

**5 · Add trap questions.** Ask things that are in no file. The right reply is "NOT FOUND". If your system guesses, you have found a real bug.

**6 · Measure retrieval separately from answers.** A no-LLM hit@4 check takes seconds and tells you whether the model ever had a chance.

TRY IT

# Run it on your laptop in about 10 minutes.

Windows PowerShell. You need Python and, for model answers, **Ollama** with `ollama pull qwen2.5-coder:7b`.

```
# get the project: github.com/SabaAutomation/multidoc-brain
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python selfcheck.py              # 29 checks, no Ollama needed
python run_experiment.py         # retrieval results in seconds
python run_experiment.py --answer  # ask the local model (slow on CPU,
resumable)
python make_report.py
python make_dashboard.py         # then open results\dashboard.html
```

### Use it on your own folder

```
python ask.py --docs "C:\path\to\your\folder" --level P3
```

Supports `.pdf .docx .xlsx .csv .md .txt`. Answers come with the source file and sheet or page. Without Tesseract installed, P3 cannot read scans and behaves like P2.

> **Tip:** run `--level P0` and `--level P3` on the same question and compare. That one comparison is the whole lesson.

BUILD YOUR OWN TEST

# 30 questions, one afternoon.

1. Pick files that look like your real work: a table, a spreadsheet, a scan.
2. Write each question **and its answer** from the file first. Keep the answer short enough to match as text.
3. Add about 10% trap questions with no answer in any file.
4. Run a plain-text baseline (P0). Write the score down.
5. Change **one** thing. Run again. Compare per file type, not just the total.

| Mix | Questions | Why |
|---|---|---|
| Prose | 10 | Your control: should pass at every level |
| Table in a Word file | 5 | Shows dropped tables |
| Spreadsheet or CSV | 5 | Shows lost headers |
| Scan or image PDF | 5 | Shows missing OCR |
| Trap questions | 3–5 | Shows guessing |

## Question templates that work

- "What is the price of [item code]?"
- "Who is the [role] named on the notice?"
- "How many days is the [policy] period?"
- Trap: "What is the [thing that appears nowhere]?"

BONUS 1

# The three-question test for any "AI can do X" claim.

**1 · Is there a test with known answers?** Pixel matches, unit tests, a question set. A screen recording is a demo, not a result.

**2 · Who ran it?** The team's own numbers are a starting point. Independent numbers are better.

**3 · Can I reproduce it?** Can you open the repo and run the test yourself?

### Red flags

- Only a success video, no failures shown.
- A percentage with no denominator.
- No "unknown" or "limits" section.
- Numbers that change between articles and nobody links the source.

### Green flags

- Evidence and limitations reported with every conclusion.
- Trap questions or negative tests.
- Raw results published next to the summary.

BONUS 2

# MCP on one page: one standard plug.

**Before MCP**

**3 × 3 = 9**

3 AI apps, 3 tools: custom code for every pair.

**With MCP**

**3 + 3 = 6**

Each side speaks the standard once.

The Model Context Protocol is an open standard for connecting AI apps to tools and data: a database, Slack, your calendar. Think USB-C, but for giving AI access. In practice an AI app implements an MCP *client* and a tool exposes an MCP *server*; the 3 + 3 count is a simplification.

## Facts, with sources

- Anthropic introduced MCP in late 2024.
- On 9 December 2025 Anthropic announced it was donating MCP to the Agentic AI Foundation, a directed fund under the Linux Foundation. Co-founders: Anthropic, Block and OpenAI.

Sources: anthropic.com/news (donating the Model Context Protocol and establishing the Agentic AI Foundation); linuxfoundation.org press release on the formation of the Agentic AI Foundation.

WHAT TO BUILD NEXT

# 10 LLM projects worth building. Pick one. Ship it this week.

10 Researcher that builds its own evidence map

9 AI that tries to break your app (autonomous QA)

8 Multi-agent debate system

7 Code reviewer that gets the intent of the ticket

6 Video memory engine

5 Second brain that argues back

4 AI that spots contradictions across documents

3 LLM memory graph

2 "Why did this happen?" root-cause agent

1 AI that learns your workflow

**Where MultiDoc Brain fits:** project 4 starts exactly where this experiment ends. Once your reader gets every table and scan into the index, the next question is which documents disagree.

**Follow @sabaautomations**

AI agents and automation, built in public · github.com/SabaAutomation/multidoc-brain